

Hand Gesture Recognition

Complete Code Example (Hand Gesture Recognition).

```
import cv2

import numpy as np


# Step 1: Capture video from webcam

cap = cv2.VideoCapture(0)


while True:

    # Step 2: Read each frame

    ret, frame = cap.read()

    if not ret:

        break


    # Step 3: Flip frame for mirror effect

    frame = cv2.flip(frame, 1)


    # Step 4: Define region of interest (ROI) for hand

    roi = frame[100:400, 100:400]

    cv2.rectangle(frame, (100, 100), (400, 400), (0, 255, 0), 2)


    # Step 5: Convert ROI to HSV color space

    hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)


    # Step 6: Define skin color range and create mask

    lower_skin = np.array([0, 20, 70], dtype=np.uint8)

    upper_skin = np.array([20, 255, 255], dtype=np.uint8)
```

Hand Gesture Recognition

```
mask = cv2.inRange(hsv, lower_skin, upper_skin)
```

```
# Step 7: Apply morphological transformations
```

```
mask = cv2.dilate(mask, np.ones((3, 3), np.uint8), iterations=4)
```

```
mask = cv2.GaussianBlur(mask, (5, 5), 100)
```

```
# Step 8: Find contours
```

```
contours, hierarchy = cv2.findContours(mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
```

```
if contours:
```

```
    cnt = max(contours, key=lambda x: cv2.contourArea(x))
```

```
    hull = cv2.convexHull(cnt)
```

```
    cv2.drawContours(roi, [hull], -1, (0, 255, 0), 2)
```

```
# Step 9: Count defects (finger gaps)
```

```
hull_indices = cv2.convexHull(cnt, returnPoints=False)
```

```
defects = cv2.convexityDefects(cnt, hull_indices)
```

```
if defects is not None:
```

```
    count_defects = 0
```

```
    for i in range(defects.shape[0]):
```

```
        s, e, f, d = defects[i, 0]
```

```
        start = tuple(cnt[s][0])
```

```
        end = tuple(cnt[e][0])
```

```
        far = tuple(cnt[f][0])
```

```
        cv2.circle(roi, far, 5, (0, 0, 255), -1)
```

Hand Gesture Recognition

```
count_defects += 1

# Step 10: Map gestures to actions

if count_defects == 0:

    cv2.putText(frame, "Medicine: Paracetamol", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0,
255), 2)

elif count_defects == 1:

    cv2.putText(frame, "Medicine: Ibuprofen", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255),
2)

elif count_defects == 2:

    cv2.putText(frame, "Medicine: Aspirin", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

# Step 11: Show frames

cv2.imshow("Frame", frame)

cv2.imshow("Mask", mask)

# Step 12: Exit on 'q'

if cv2.waitKey(1) & 0xFF == ord('q'):

    break

cap.release()

cv2.destroyAllWindows()
```

Line-by-Line Explanation

- **Step 1–2:** Capture video stream from webcam.

Hand Gesture Recognition

- **Step 3:** Flip frame for natural mirror-like interaction.
- **Step 4:** Define ROI (region of interest) where hand gestures are expected.
- **Step 5–6:** Convert ROI to HSV color space and apply skin color thresholding.
- **Step 7:** Clean mask using dilation and Gaussian blur.
- **Step 8:** Find contours to detect hand shape.
- **Step 9:** Use convex hull and convexity defects to detect finger gaps.
- **Step 10:** Map number of defects (finger count) to specific medicines.
- **Step 11–12:** Display results and exit gracefully.

Pull Report (Interview Explanation)

When asked in an interview, you can explain your project like this:

- **Problem Statement:** During COVID-19, shopkeepers needed a contactless way to identify medicines.
- **Solution:** Built a hand gesture recognition system using OpenCV and Python.
- **Implementation:**
 - Captured video using OpenCV.
 - Applied skin color detection and contour analysis.
 - Used convex hull defects to count fingers.
 - Mapped gestures to medicines.
- **Impact:** Reduced physical contact, improved safety, and demonstrated practical computer vision application.

Challenges & How You Handled Them

Hand Gesture Recognition

Challenge	How You Handled It
Lighting variations	Used HSV color space instead of RGB for better skin detection.
Background noise	Restricted detection to ROI and applied Gaussian blur.
Gesture misclassification	Added convex hull + defect counting for accuracy.
Performance issues	Optimized mask operations and limited ROI size.

Deep Explanation →

- 🔗 **Video Capture** (`cv2.VideoCapture(0)`) Opens your webcam. 0 means default camera. If you had multiple cameras, you'd use 1, 2, etc.
- 🔗 **Frame Flip** (`cv2.flip(frame, 1)`) Flips the image horizontally so it feels like a mirror. Without this, your hand movement looks reversed.
- 🔗 **ROI (Region of Interest)** You define a box (`frame[100:400, 100:400]`) where you expect the hand. This reduces noise and speeds up processing.
- 🔗 **HSV Conversion** (`cv2.cvtColor`) Converts the image from BGR (default) to HSV. HSV is better for detecting skin tones under different lighting.
- 🔗 **Skin Mask** (`cv2.inRange`) Creates a binary mask where pixels in the skin color range are white, others are black.
- 🔗 **Morphological Ops** (`cv2.dilate`, `cv2.GaussianBlur`) Smooths the mask, removes small noise, and makes contours cleaner.
- 🔗 **Contours** (`cv2.findContours`) Finds the outline of your hand. The largest contour is assumed to be your hand.
- 🔗 **Convex Hull & Defects** (`cv2.convexHull`, `cv2.convexityDefects`) Convex hull wraps your hand like a rubber band. Defects are the gaps between fingers — counting them helps recognize gestures.
- 🔗 **Gesture Mapping** (if `count_defects == 0`: **etc.**) Based on finger count, you map gestures to medicines.

Challenge	Why It Happened	How You Solved It
-----------	-----------------	-------------------

Hand Gesture Recognition

Lighting variations	Skin detection failed in dark or bright light.	Used HSV color space instead of RGB, adjusted thresholds dynamically.
Background noise	Other objects confused the system.	Restricted detection to ROI and applied Gaussian blur.
Hand not detected	ROI too small or hand outside box.	Enlarged ROI and guided users to keep hand inside green box.
Gesture misclassification	Fingers overlapped or detection unstable.	Added convex hull + defect counting for more reliable classification.
Performance issues	Processing full frame slowed system.	Limited ROI size and optimized mask operations.
Camera not showing output in VS Code	OpenCV GUI windows don't always render inside VS Code terminal.	Ran script from external terminal (python hand_gesture.py) instead of VS Code's internal console.

Technical Questions

- Q:** Why did you choose OpenCV for this project? **A:** OpenCV is lightweight, open-source, and optimized for real-time computer vision tasks. It provides built-in functions for video capture, contour detection, and image processing, which made it ideal for implementing gesture recognition quickly and efficiently.
- Q:** How does your system detect hand gestures? **A:** The system captures video frames, converts them to HSV color space, applies a skin color mask, and finds contours. Using convex hull and convexity defects, it counts finger gaps to classify gestures. Each gesture is mapped to a specific medicine.
- Q:** Why did you use HSV instead of RGB? **A:** HSV separates color information from brightness, making skin detection more robust under varying lighting conditions. RGB is sensitive to shadows and brightness changes, which caused misclassification during testing.

Hand Gesture Recognition

4. **Q:** How do you handle background noise? **A:** I restricted detection to a Region of Interest (ROI) and applied Gaussian blur and morphological operations to clean the mask. This reduced false positives from background objects.
5. **Q:** What challenges did you face with gesture recognition? **A:**
 - Lighting variations → solved with HSV thresholds.
 - Background interference → solved with ROI restriction.
 - Misclassification when fingers overlapped → solved using convex hull defect analysis.
 - Performance issues → solved by limiting ROI size and optimizing mask operations.

Application & Impact Questions

6. **Q:** What was the motivation behind this project? **A:** During COVID-19, shopkeepers needed a contactless way to identify medicines. My system allowed them to use hand gestures instead of touching surfaces, reducing physical contact and improving safety.
7. **Q:** How scalable is your solution? **A:** The system can be extended by mapping more gestures to medicines or integrating a database. It could also be adapted for other applications like gesture-based menu navigation or contactless kiosks.
8. **Q:** How would you improve this project in the future? **A:**
 - Integrate machine learning models (like CNNs) for more accurate gesture classification.
 - Add voice feedback for accessibility.
 - Use depth cameras for better hand segmentation.
 - Build a GUI for shopkeepers to customize medicine mappings.

Hand Gesture Recognition

Behavioral Questions

9. **Q:** Tell me about a time you faced a major challenge in this project. **A:** Initially, the system failed under poor lighting, and gestures weren't recognized. I researched HSV color ranges, experimented with thresholds, and tested under different conditions until I found a reliable range. This taught me persistence and problem-solving.
10. **Q:** What did you learn from this project? **A:** I learned how to apply computer vision concepts in real-world scenarios, how to debug issues systematically, and how to balance accuracy with performance. Most importantly, I learned to adapt solutions to practical constraints like lighting and background noise.